

Rapport SAE S41 : Projet 2048

Partie 1

Nous avons d'abord retiré tous les mécanismes de synchronisation (mutex, variables conditionnelles). A la place on a utilisé des signaux, des flags volatile `sig_atomic_t` pour l'attente active et on a mis en place une architecture en cascade : le processus principal ne réveille que le thread Move. Une fois le calcul terminé et la grille modifiée, Move réveille le thread Goal, qui effectue l'affichage et libère ensuite le processus principal. C'est exactement ce qui était fait par les mécanismes de synchronisation de façon beaucoup plus simple et sans attente active, mais on a pas de section critique alors c'était **inutile**.

Pour implémenter le nouveau joueur et abandonner l'état de jeu global unique, nous avons encapsulé les données relatives à un joueur dans une nouvelle structure `ClientSession`. Cette structure regroupe l'identifiant du processus d'entrée (`input_pid`), celui de son affichage dédié (`display_pid`), le descripteur du pipe d'affichage (`display_fd`), ainsi que l'état de sa grille de jeu (`GameState`). Pour gérer dynamiquement le nombre variable de joueurs connectés, nous avons réutilisé le module `array_list` (du module P31). Cela nous a permis d'instancier une liste dynamique de sessions dans le processus principal, facilitant l'ajout de nouveaux joueurs lors de la réception d'une commande `CMD_HANDSHAKE`.

```
typedef struct ClientSession
{
    pid_t input_pid;
    pid_t display_pid;
    int display_fd;
    GameState state;
} ClientSession;
```

Partie 2

** Segment de Mémoire Partagé : SMP*

L'étape 2 impose de déporter l'intégralité de la logique de calcul (déplacement des tuiles, calcul du score et vérification des conditions de victoire/défaite) au sein d'un Segment de Mémoire Partagée (SMP), tout en conservant le stockage persistant des différentes sessions multijoueurs dans le tas (Heap) du processus principal.

1. Architecture Technique et Pipeline de Données

Nous avons mis en place un SMP unique, dimensionné pour accueillir non pas un simple GameState, mais une structure complète nommée SharedGameSlot. Cette structure agit comme un sas de traitement contenant l'état du jeu, la commande utilisateur, les descripteurs de communication et un statut d'avancement (SlotStatus).

```
/* Structure du segment de mémoire partagé */
typedef struct SharedGameSlot
{
    GameState state;
    UserCommand cmd;
    int display_fd;
    pid_t input_pid;
    SlotStatus status;
} SharedGameSlot;

/* Statut actuel de l'emplacement de mémoire partagé */
typedef enum
{
    SLOT_FREE,
    SLOT_IDLE,
    SLOT_TO_MOVE,
    SLOT_TO_GOAL
} SlotStatus;
```

A. Modélisation des Données (Séparation Tas / SMP)

- **La structure de persistance : ClientSession (Allouée sur le Tas)**
Cette structure sert de sauvegarde "froide" pour maintenir l'historique de toutes les parties en parallèle. Stockée dynamiquement dans la liste `array_list` players gérée par le processus Main, elle conserve les identifiants de communication du joueur (`input_pid`, `display_fd`) et la sauvegarde complète de sa grille (`GameState`) en attendant son prochain coup.
- **Le sas de traitement : SharedGameSlot (Alloué dans le SMP)**
Dimensionné pour accueillir les données d'une seule partie à un instant T, ce segment agit comme un espace de travail exclusif. Il rassemble tout le contexte nécessaire aux threads de calcul pour travailler de manière autonome : la copie de travail de la grille, la commande utilisateur (`cmd`), les descripteurs de communication transférés temporairement, et un statut d'avancement (`SlotStatus` pouvant valoir `SLOT_FREE`, `SLOT_TO_MOVE` ou `SLOT_TO_GOAL`).

B. Le Pipeline de Données

Le flux de données à travers ces deux structures suit un pipeline d'exécution en trois phases :

- **Extraction et Transfert (Aller) :** Lorsqu'une commande est reçue via le pipe nommé, le thread Main identifie la session correspondante dans la liste dynamique players (sur le tas). Il attend que le SMP soit déclaré libre (`SLOT_FREE`), y copie l'état de jeu de ce joueur, ainsi que la commande à exécuter, puis passe le statut à `SLOT_TO_MOVE`.
- **Traitement en cascade (SMP) :** Les threads de calcul prennent le relais et travaillent exclusivement sur les adresses du SMP. Le thread Move exécute `move_grid`, puis passe le relais en modifiant le statut à `SLOT_TO_GOAL`. Le thread Goal prend la suite pour vérifier les conditions de fin avec `check_win` et `check_lose`.
- **Rapatriement (Retour) :** Contrairement à l'approche initiale, ce n'est pas le thread Main qui rapatrie les données, mais le thread Goal. Une fois l'affichage mis à jour via le pipe du client, le thread Goal copie le `GameState` mis à jour depuis le SMP pour écraser l'ancien état dans le tas du processus principal. Il signale ensuite au client qu'il peut rejouer, et libère le SMP en le repassant à `SLOT_FREE`.

2. Synchronisation à Double Verrouillage

Le SMP étant une ressource critique globale (un goulot d'étranglement par lequel passent tous les joueurs), une exclusion mutuelle stricte était nécessaire pour éviter la corruption croisée des grilles.

Nous avons implémenté un système reposant sur deux Mutex distincts et des Sémaphores :

- **shm_mutex** : Protège l'accès au segment de mémoire partagée. Il est couplé aux sémaphores `sem_move`, `sem_goal` et `sem_free`. Cela permet de réveiller uniquement le thread concerné par l'étape de calcul en cours (concept de machine à états), évitant ainsi toute attente active ou condition de course.
- **heap_mutex** : Un second verrou indispensable ajouté pour protéger le tableau dynamique `players`. Sans ce verrou, si le processus principal insère un nouveau joueur (provoquant un `realloc` de la liste) au moment même où le thread `Goal` tente de rapatrier les données d'un autre joueur, une erreur de segmentation se produirait.

3. Sécurité, Fuites et Cas aux Limites

Déporter la logique dans des processus et threads indépendants a nécessité la sécurisation de l'architecture contre plusieurs vulnérabilités système critiques que nous avons corrigées :

- **Protection SIGPIPE** : Si le terminal d'affichage d'un client est fermé brutalement, l'écriture dans son pipe anonyme brisé génère un signal `SIGPIPE`. Nous avons forcé le système à l'ignorer (`SIG_IGN`) afin d'empêcher le crash immédiat du moteur central.
- **Processus Zombies et Descripteurs de fichiers** : Lors de la déconnexion d'un client (`CMD_QUIT`), le serveur ferme son descripteur de fichier (ce qui envoie un EOF propre au processus d'affichage), puis utilise `waitpid` pour collecter le code de retour de l'enfant, évitant ainsi l'accumulation de processus zombies et les fuites de File Descriptors.
- **Interblocages (Deadlocks) à la fermeture** : Pour prévenir la situation où un joueur se retrouverait bloqué indéfiniment en cas d'interruption abrupte du serveur (via `CTRL+C`), la phase de nettoyage diffuse un signal de réveil `SIG_CLEAN_EXIT` à tous les clients encore connectés avant de libérer la mémoire partagée et de s'éteindre.

Partie 3

Pour l'étape 3, la taille du segment de mémoire partagée (SMP) est définie dynamiquement au lancement du processus principal (game_main) via l'argument N (ex: ./game_2048 N). Le segment ne contient plus une structure isolée, mais un tableau contigu de structures : GameState shared_area[N].

Chaque instance de ClientSession (stockée dans notre array_list sur le tas) s'est vue attribuer un attribut supplémentaire : un identifiant unique slot_index (allant de 0 à N-1).

```
typedef struct ClientSession
{
    pid_t input_pid;
    pid_t display_pid;
    int display_fd;
    GameState state;
    int slot_index;
} ClientSession;
```

Mécanique de fonctionnement :

Lorsqu'un nouveau joueur se connecte (réception d'un CMD_HANDSHAKE), le thread principal parcourt le SMP pour lui réserver un emplacement libre. Cet slot_index est ensuite utilisé pour calculer l'offset en mémoire physique : adresse_base_smp + (slot_index * sizeof(GameState)), permettant un accès direct en O(1). De plus, chaque joueur possède son propre thread "move" et thread "goal" car nous voulions simuler un vrai multijoueur, on peut donc traiter les inputs de tous les joueurs en parallèle (même si c'est humainement impossible que tous les joueurs jouent un coup en même temps puisque le jeu est "multijoueur" sur une seule machine, donc un seul clavier, ce qui n'a absolument aucun intérêt)

Synchronisation et Concurrency :

Pour éviter qu'un calcul sur une grille ne bloque l'ensemble du serveur (goulot d'étranglement), nous avons fait évoluer notre mécanisme de synchronisation :

- **Verrouillage par Slot** : Au lieu d'utiliser un unique mutex global, nous avons mis en place un tableau de verrous. Ainsi, les opérations d'exclusion mutuelle

sont limitées au slot_index concerné. Les requêtes de joueurs occupant des slots différents peuvent donc être traitées en véritable simultanéité.

- **Passage de contexte** : Les routines de calcul (thread_move_routine et thread_goal_routine) reçoivent désormais le slot_index en paramètre. Elles savent ainsi exactement sur quelle zone mémoire du SMP appliquer la logique (move_grid, check_win, check_lose) sans interférer avec les autres parties en cours.

Défis techniques rencontrés et solutions apportées :

1) Interblocage lié à l'héritage des descripteurs de fichiers (FD Leak)

Problème : Lorsqu'un nouveau joueur se connectait, le serveur utilisait fork() puis execl() pour lancer son processus d'affichage. Cependant, l'enfant héritait d'une copie de tous les descripteurs de fichiers (pipes) des joueurs précédents. Si un ancien joueur quittait la partie, son processus d'affichage ne se terminait jamais car le pipe n'envoyait pas de signal EOF (le nouveau processus d'affichage gardant une copie fantôme du descripteur ouverte). Cela bloquait indéfiniment le serveur sur la fonction waitpid().

Solution : Nous avons implémenté un nettoyage manuel strict. Avant de faire appel à execl(), le processus enfant parcourt la liste des joueurs actifs et force la fermeture (close()) de tous les descripteurs de fichiers qui ne lui appartiennent pas.

Bilan global :

Grâce à l'architecture que nous avons pu mettre en place et suite à de nombreux tests que nous avons effectués tout au long du développement de notre application, nous n'avons, à ce jour, découvert aucune fonctionnalité ne fonctionnant pas. En effet, nous sommes parvenu implémenter le système de multijoueur pour N joueurs en se servant d'un segment de mémoire partagé qui est protégé par des mutex et des sémaphores, protégeant ainsi le jeu de toute erreurs de segmentation, d'interblocages ou de pertes de données.

Répartition des tâches :

- **Valentin** : Partie 3, rédaction du rapport et tests.
- **Thomas** : Partie 1, rédaction du rapport et tests.
- **Diego** : Partie 2, rédaction du rapport, tests et nettoyage du code.